Laboratorio 3: Sistema de Gestión de Inventario Seguro

1. Introducción

Este laboratorio se enfoca en la transición hacia la **concisión moderna** y la **seguridad estructural** de Kotlin.

2. Objetivos de Aprendizaje

- Implementar el sistema de tipos de Kotlin, diferenciando entre variables mutables (var), inmutables (val) y tipos anulables (?).
- Dominar los operadores de seguridad nula (**Elvis operator ?:**) y el manejo de **plantillas de cadena** para generar reportes dinámicos.
- Aplicar estructuras de control como expresiones para determinar estados lógicos del negocio.

3. Contexto

En el desarrollo profesional, la estabilidad de una App depende de cómo manejamos la ausencia de datos (nulos). Mientras que en Java un olvido causaba un cierre inesperado (**NullPointerException**), en Kotlin la seguridad es un contrato de "ley" integrado en el compilador. Aprenderemos a crear código robusto que detecte errores antes de que la aplicación llegue al usuario.

4. Preparación del Entorno (Basado en Lab 1 y 2)

Para este laboratorio, los estudiantes deben trabajar en su proyecto estandarizado en la **Semana 1 (seguir las instrucciones del paso 1 del Laboratorio 2)**:

- **Clonación**: En el explorador de archivos, copiar la carpeta del Laboratorio 1 y cambiar el nombre a **RDA1_Laboratorio3_Inventario_NombreApellido**.
- **Limpieza**: Borrar las carpetas ocultas **.gradle** y **build** dentro de la nueva carpeta para evitar conflictos de caché.
- **Renombrado**: Abrir la carpeta en Android Studio Ladybug. Ir al archivo **settings.gradle.kts** y cambiar el nombre del proyecto: **rootProject.name = "InventarioSeguro"**
- **Sincronización**: Hacer clic en el icono del elefante (**Sync Project with Gradle Files**) en la esquina superior derecha y espera a que la barra de progreso termine.

5. Paso a Paso Detallado

Paso 0: Creación del Archivo de Lógica

No usaremos el archivo de la interfaz todavía. Crearemos un archivo de Kotlin puro.

1. En el panel izquierdo Project, navegar a: **app/src/main/java/[su.paquete.de.la.semana.anterior]/**.
2. Hacer clic derecho sobre la carpeta de su paquete **-> New -> Kotlin Class/File**.
3. Seleccionar File (no Class) y escribir el nombre: **InventoryManager**. Presionar Enter.
4. **Android Studio** creará un archivo vacío llamado **InventoryManager.kt**.

Paso 1: Definición del Modelo de Producto (Seguridad de Tipos)

Definiremos qué campos son obligatorios y cuáles opcionales. En Kotlin, por defecto, nada es nulo a menos que lo autoricemos con **?**. Copiar el siguiente bloque de código dentro de InventoryManager.kt. Este código sustituirá cualquier contenido previo en ese archivo.

```
fun main() {
    // Definición de variables de producto
    val productId: Int = 101                  // ID inmutable (no cambia)
    val productName: String = "Smartphone X1" // Nombre obligatorio (no nulo)
    var productDescription: String? = null    // ? permite que sea nulo
    var stockQuantity: Int = 25              // Cantidad mutable (cambia con ventas)
    val unitPrice: Double = 599.99           // Precio base inmutable.

    // El compilador de Java 21/Kotlin 2.0 gestiona aquí la memoria eficientemente.
    println("--- Datos Iniciales Cargados ---")
}
```

- Por qué: Usamos **fun main()** porque permite ejecutar lógica de Kotlin rápidamente sin necesidad de desplegar toda la interfaz de Android en el emulador, ahorrando tiempo de compilación.

Paso 2: Lógica de Precios con IVA

Calcularemos el precio final. Usamos **const val** para valores que conocemos en tiempo de compilación. Debajo del último **println** dentro de la función **main**, añadir el cálculo del impuesto:

```
    // 2. Lógica de Impuestos
    val TAX_RATE = 0.15 // IVA del 15%
    val totalPriceWithTax = unitPrice * (1 + TAX_RATE)

    // Nota: No puede modificar 'unitPrice' porque es 'val'.
    // Si lo intenta, Android Studio marcará un error rojo.
```

Paso 3: Manejo Seguro de Nulos (Operador Elvis)

Evitaremos que la App muestre "null" al usuario. El operador **Elvis (?:)** proporciona un valor de respaldo. Ahora, añadir la lógica para manejar la descripción que definimos como nula (null). Pegarlo a continuación en el mismo archivo:

```
    // 3. Uso del Operador Elvis (?:)
    // Si productDescription es null, se asigna el texto de la derecha.
    val descriptionToShow = productDescription ?: "Sin descripción
disponible"

    println("Producto: $productName")
    println("Descripción: $descriptionToShow")
```

Paso 4: Ejecución y Validación de Resultados

A diferencia de los laboratorios anteriores, no necesita presionar el botón "Run" (flecha verde) de la barra superior, ya que eso lanzaría el emulador.

- Buscar el icono verde de "**Play**" (triángulo pequeño) que aparece justo a la izquierda de la línea **fun main() {** en el editor de código.
- Hacer clic en él y selecciona **Run 'InventoryManagerKt'**.
- Se abrirá la pestaña **Run** en la parte inferior de Android Studio. Deberá ver los primeros mensajes impresos.

Paso 5: Actualización de Inventario

Simularemos una transacción. Es vital validar que el estado del sistema sea coherente. Para completar el flujo, añadir este último bloque antes del cierre de la llave **}** de la función **main**:

```
    // 4. Simulación de Venta
    val itemsSold = 5
    stockQuantity -= itemsSold // Actualización de variable mutable

    // 5. Generación de Reporte con Plantilla de Cadena
    val report = """
        --- REPORTE DE INVENTARIO ---
        ID: $productId | Nombre: $productName
        Precio Final: $${String.format("%.2f", totalPriceWithTax)}
        Stock Actual: $stockQuantity
        Estado: ${if (stockQuantity > 10) "Suficiente" else "Crítico"}
        ----------------------------
    """.trimIndent()

    println(report)
```

Paso 6: Ejecución Final

- Vuelva a hacer clic en el **triángulo verde** junto a **fun main()**.
- Verificar que en la consola inferior aparezca el reporte formateado correctamente.
- **Sincronización final**: Presionar **Ctrl + S** (o **Cmd + S**) para guardar. No es necesario volver a sincronizar Gradle a menos que modifique el archivo **build.gradle**.

6. Resultados Esperados

Al ejecutar el código, la consola de **Android Studio** debe mostrar:

```
--- Datos Iniciales Cargados ---
Producto: Smartphone X1
Descripción: Sin descripción disponible
--- REPORTE DE INVENTARIO ---
ID: 101 | Nombre: Smartphone X1
Precio Final: $689.99
Stock Actual: 20
Estado: Suficiente
-----------------------------
```

7. Desafío Extra (Análisis Crítico)

Desafío 1: Asignar null a productName

¿Qué pasaría si intenta asignar null a productName? Intentar escribir productName = null y observar el mensaje del compilador. Explicar por qué esto protege la base de datos de su aplicación.

Desafío 2: El Validador de Precios Dinámico

En las aplicaciones reales, los datos no siempre vienen perfectos. A veces, un usuario o una API externa puede enviar un texto en lugar de un número. Su misión es hacer que el sistema sea "indestructible" ante datos erróneos.

Instrucciones del Reto:

- **Entrada de Usuario Simulada:** Declarar una nueva variable llamada inputUsuario: String? = "650.50". Esta variable representa lo que alguien escribió en la pantalla del celular.
- **Conversión Segura**: Utilizar la función de Kotlin .toDoubleOrNull() para intentar convertir ese texto a un número real.
- **Lógica de Respaldo (Elvis Operator):**

1. Si la conversión es exitosa, usar ese nuevo precio para recalcular el **totalPriceWithTax**.
2. Si la conversión falla (por ejemplo, si el usuario escribió "Gratis"), el sistema deberá asignar automáticamente un precio de **0.0** para evitar que la App se cierre.

- Prueba: * Ejecutar el código con inputUsuario = "650.50".
    1. Luego, cambiar el valor a inputUsuario = "No quiero pagar" y volver a ejecutar.

8. Entregables

- **Informe:** con capturas de pantallas pertinentes, con mínimo las siguientes capturas:
    - Captura de pantalla de la terminal de Android Studio mostrando el **Build Successful** y el reporte generado.
    - Captura (Entorno): Una imagen de la pantalla completa de Android Studio donde se vea el archivo InventoryManager.kt abierto y la pestaña Run inferior con el reporte de inventario generado exitosamente.
    - Captura (GitHub): El historial de Commits en su cuenta de GitHub, donde se vea claramente el registro del Laboratorio 3 y ahora el nuevo commit de la Semana 2.
    - Captura (Git Log): Una imagen de la pestaña Git dentro de Android Studio (History) mostrando la "Línea de Tiempo" de sus cambios.
- Breve explicación (máximo 100 palabras) sobre la diferencia entre **val** y **var** en este contexto de inventario.
- **Enlace (URL):** El link público a su repositorio de GitHub para revisión del docente.

9. RÚBRICA DE EVALUACIÓN

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.